

mechArm-270 M5



1 Product introduction

mechArm 270 M5 belongs to the "mechArm" series of small six-arm robotic arms of Elephant Robotics. M5Stack-basic is used as the main control and supports ROS simulation software. It is an industrial-like configuration launched by Elephant Robotics for maker innovation and robot industry-university-research services Small six-axis robotic arm.

The weight of the mechArm 270 M5 is 1kg, the load is 250g, and the working radius is 270mm. The design is compact and portable, small but powerful, easy to operate, and can work with people safely. As the first small six-axis robotic arm of Elephant Robotics, it has three advantages of ease of use, safety and economy, and is a cost-effective choice.

2 Product performance

- **Classic industrial configuration, the first choice for robot industry, academia and research**

- The most classic six-axis centrosymmetric structure of industrial robots, compact and robust.
- The preferred platform for universal vocational education, colleges and individual development, applying what you have learned to break through the barriers of production, education and research.
- **Embedded Arduino ecology, infinite possibilities for dual-screen development**
 - Support Arduino rich software application ecology.
 - The fuselage carries two display screens and supports fastLED library, which is convenient for expanding application interactive output.
- **Highly open source, compatible with massive software and API**
 - Compatible with a large number of API software, built-in ROS/Moveit to simulate the running state of the robotic arm, super extensibility
 - From drag teaching and Blockly visual programming to industrial practical operation platform, one machine opens up the whole chain of production, education and research.
- **High configuration, with Lego interface**
 - Contains 6 high-performance servo motors, with fast response, small inertia and smooth rotation.
 - The base and the end are equipped with Letech interfaces, which are used in the development of various miniature embedded devices.

mechArm-270 M5



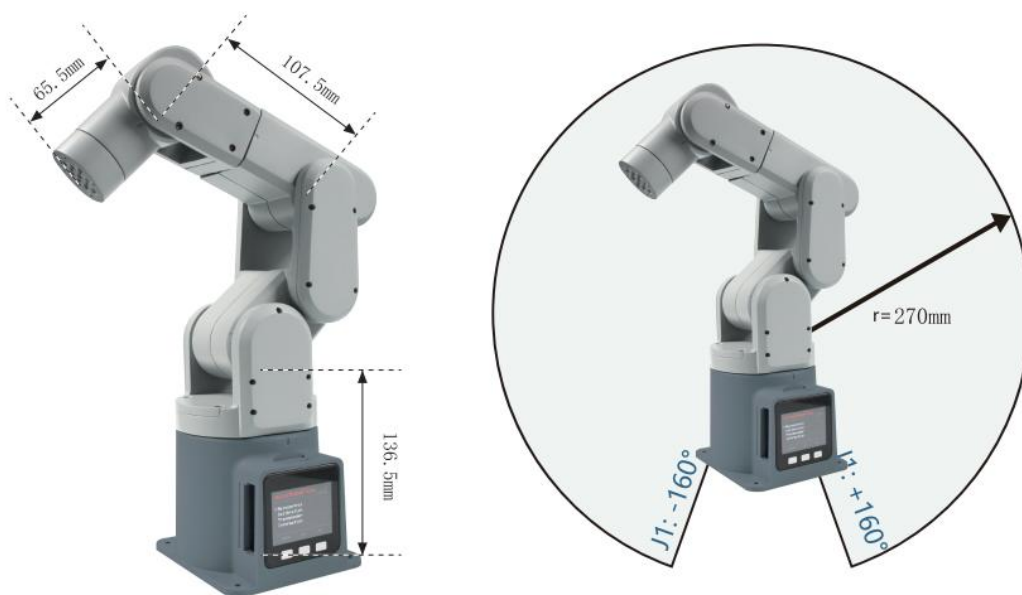
1 Structural parameters

1.1 Robot arm parameters

index	parameter
Name	Elephant small six-axis robotic arm
Model	mechArm-270 M5
Degree of freedom	6
Repeated Positioning Accuracy	± 0.5 mm
Load	250g
Dead weight	1kg

index	parameter
Effective working radius	270mm
Power input	8~12V 5A
Motor type	High-precision magnetic encoder motor * six
Movement maximum speed	120°/s
control	M5Stack-basic
communication	USB / Type-C

1.2 Scope of work



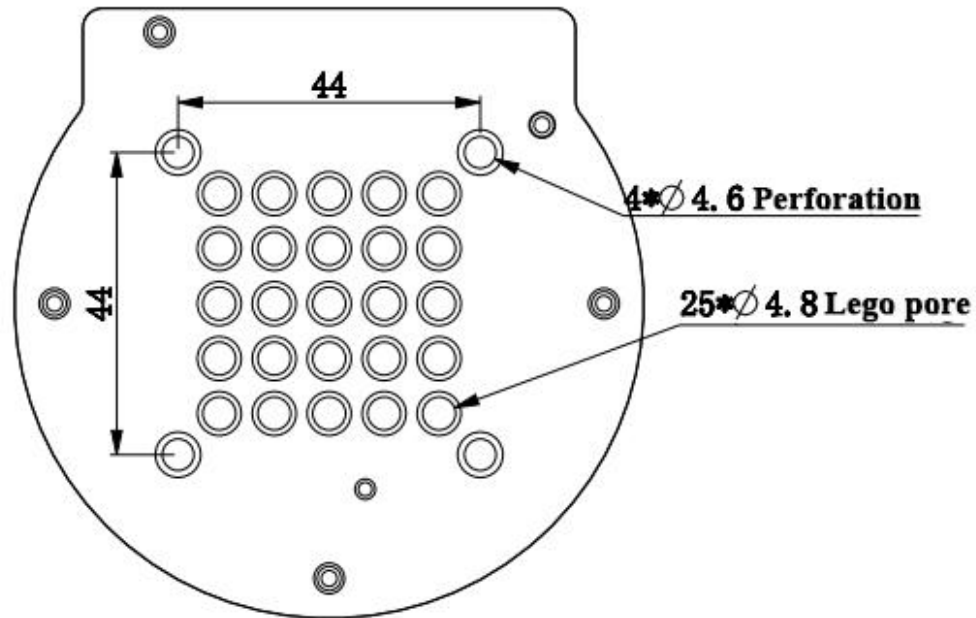
1.3 Range of motion of joints

joint	scope
J1	-160 ~ +160
J2	-85 ~ +90
J3	-180 ~ +45
J4	-160 ~ +160
J5	-100 ~ +100
J6	$-\infty \sim +\infty$

1.4 Hole installation

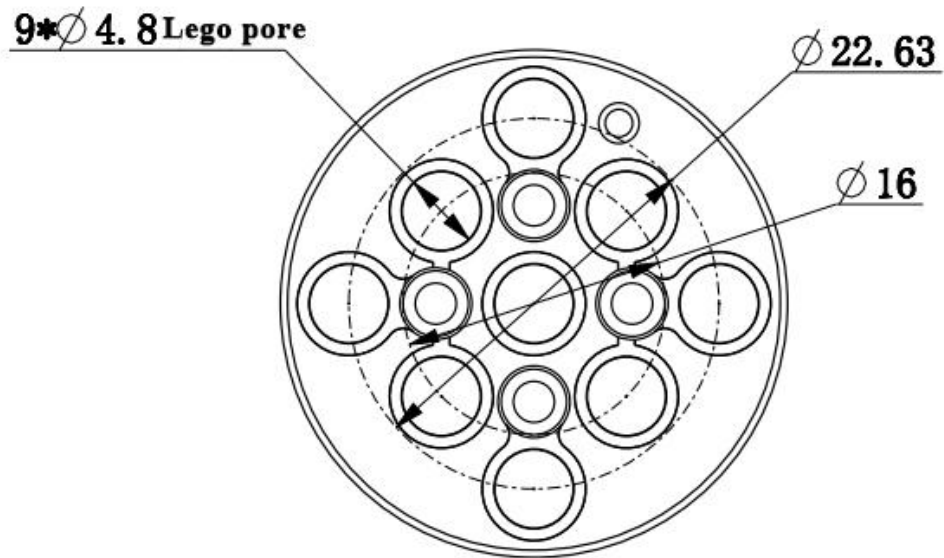
- **Hole installation**

The base is compatible with both LEGO parts installation and M4 screw installation.

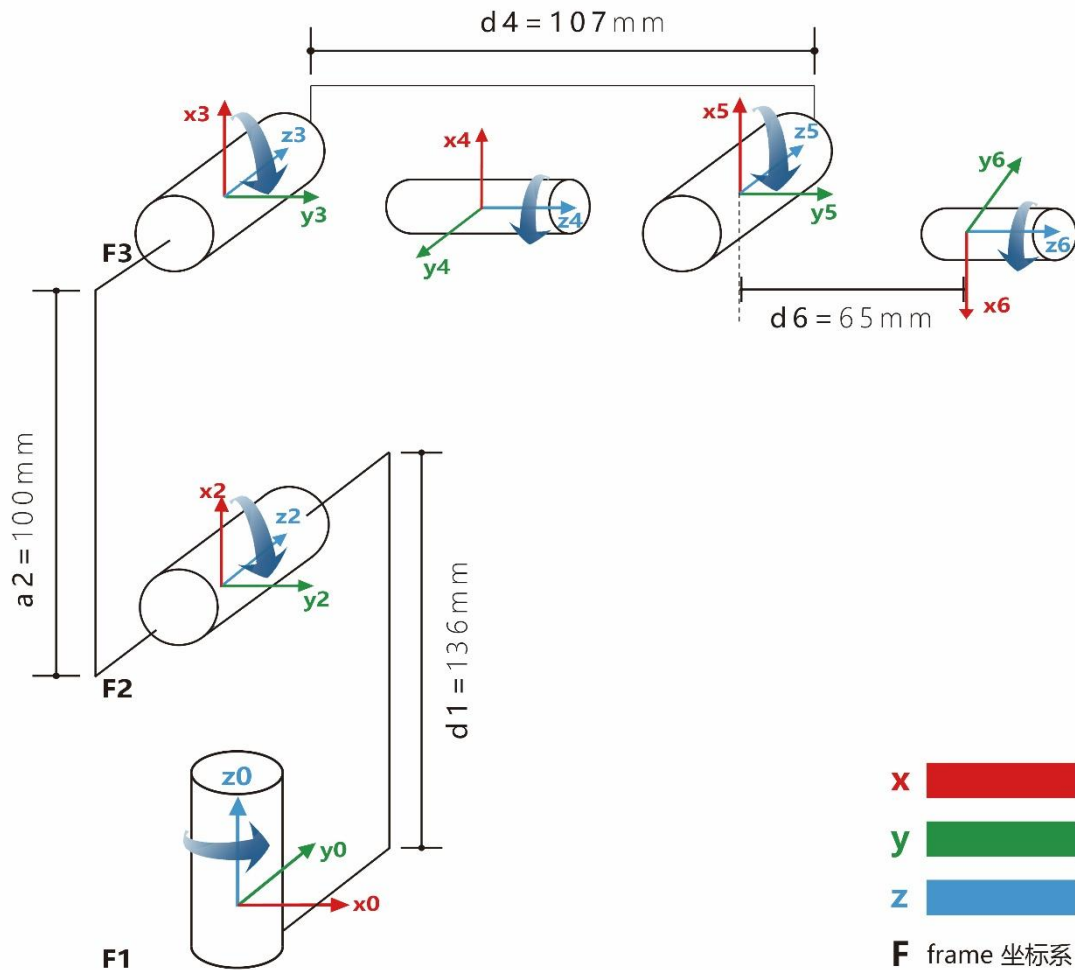


- Robot end mounting flange

end of the arm is compatible with both LEGO component holes and screw threaded holes.



1.5 DH parameters



Mechanical arm electrical interface

1 Base electrical interface

1.1 Base Introduction

A, A. The interface, display and buttons on the top of the base are shown in Figure 1-1:

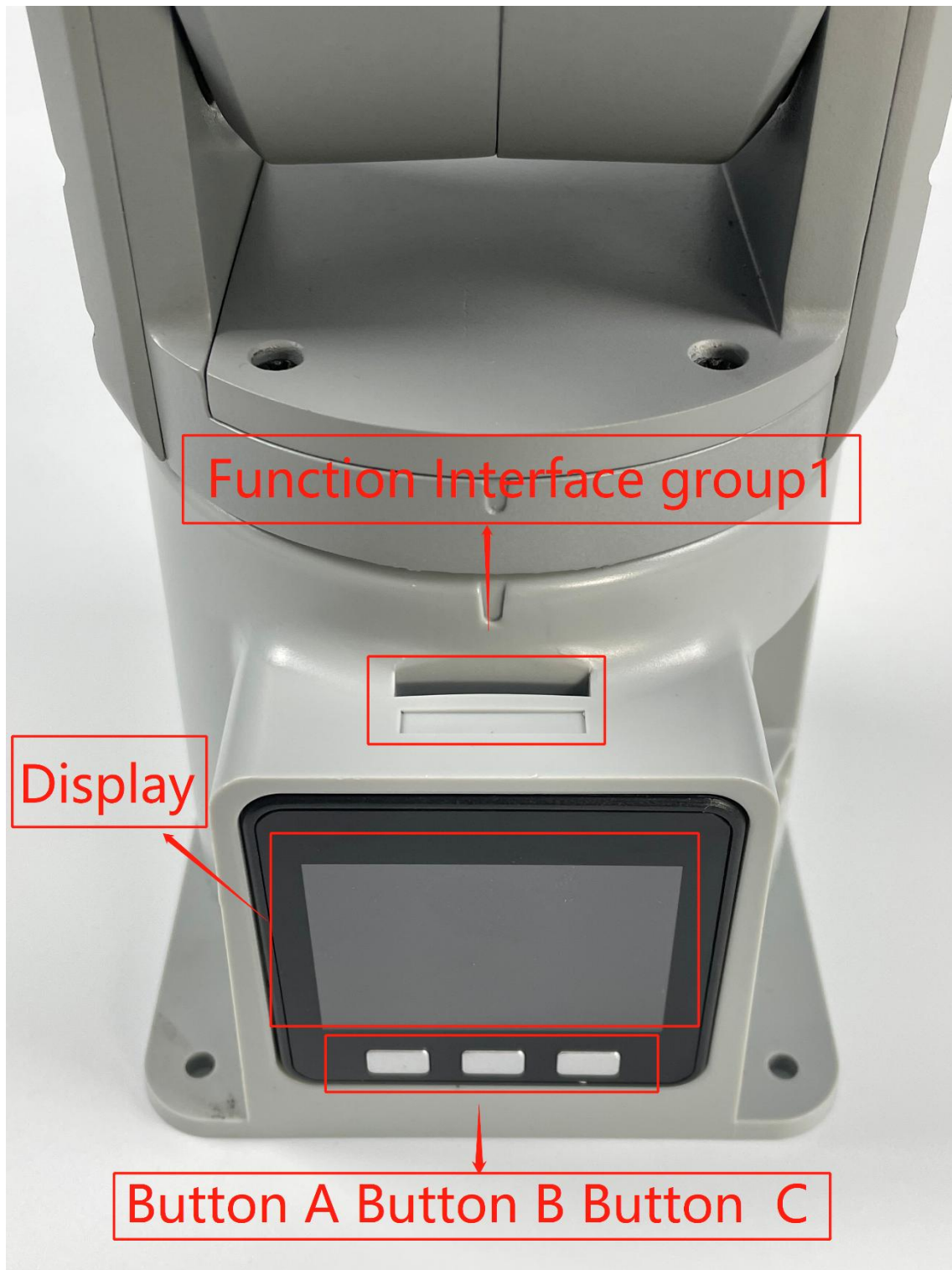


Figure 1-1 Front view of the base

B, the interface on the left side of the base is shown in Figure 1-2:



Figure 1-2 Left side view of the base

C, the interface on the right side of the base is shown in Figure 1-3:

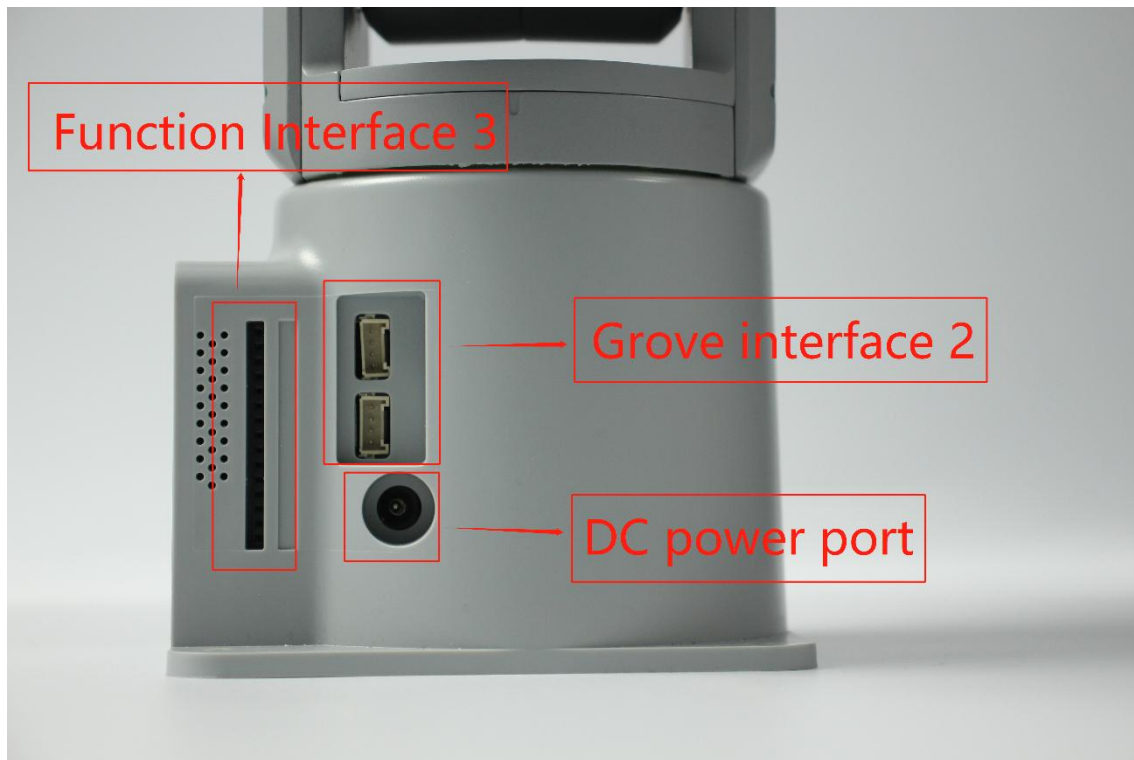


Figure 1-3 Right side view of the base

1.2 Base interface description

Note: The functional interface groups are all 2.54mm DuPont interfaces, and 2.54mm DuPont cables can be used externally;

A. Table 1-1 shows the definition of each interface in a functional interface group 1.

Tag	Signal	Function	Note
18	G18	-	Temporarily not open
19	G19	3.3 V-out-PNP output /3.3 V-int input	
23	G23	-	Temporarily not open

Tag	Signal	Function	Note
22	G22	3.3 V-out-PNP output /3.3 V-int input	
21	G21	3.3 V-out-PNP output /3.3 V-int input	
G	GND	Motherboard power signal ground	
3V3	3V3	DC3.3 V power supply	
5V	5V	Power supply, DC5V	

Table 1-1 Functional interface group 1

B. The definitions of interfaces in functional interface group 2 are the same as those in functional interface group 3, as shown in Table 1-2.

Tag	Signal	Function	Note
3	G3	3.3 V-out-PNP output /3.3 V-int input	This parameter is unavailable when usB-Type-c is used
1	G1	3.3 V-out-PNP output /3.3 V-int input	This parameter is unavailable when usB-Type-c is used
16	G16	-	Temporarily not open
17	G17	-	Temporarily not open
2	G2	3.3 V-out-PNP output /3.3 V-int input	

Tag	Signal	Function	Note
5	G5	3.3 V-out-PNP output /3.3 V-int input	
25	G25	3.3 V-out-PNP output /3.3 V-int input	
26	G26	3.3 V-out-PNP output /3.3 V-int input	
35	G35	3.3 V-int input	
34	G34	3.3 V-int input	
RST	RST	Master reset	
BAT	BAT	-	Temporarily not open
3V3	3V3	DC3.3 V power supply	
5V	5V	Power supply, DC5V	
G	GND	Motherboard power signal ground	

Table 1-2 Functional interface groups 2 and 3

Description: Figure 1-5 shows other functions of the interface. If other functions are used, the I/O function is unavailable.

GPIO TYPE	Analog Function	M-BUS			Analog Function	GPIO TYPE	
		LINE 0		LINE 1			
		GND	ADC	G35	ADC1_CH7	I	
		GND	ADC	G36	ADC1_CH0	I	
		GND	RST	EN			
I/O/T		G23	MOSI	DAC/SPK	G25	ADC2_CH8	I/O/T
I/O/T		G19	MISO	DAC	G26	ADC2_CH9	I/O/T
I/O/T		G18	SCK	3.3V			
I/O/T		G3	RXD1	TXD1	G1		I/O/T
I/O/T		G16	RXD2	TXD2	G17		I/O/T
I/O/T		G21	SDA	SCL	G22		I/O/T
I/O/T	ADC2_CH2/T2	G2	GPIO	GPIO	G5		I/O/T
I/O/T	ADC2_CH5	G12	IIS_SK	IIS_WS	G13	ADC2_CH4/T4	I/O/T
I/O/T	ADC2_CH3/T3	G15	IIS_OUT	IIS_MK	G0	ADC2_CH1/T1	I/O/T
			IIS_IN	G34	ADC1_CH6	I	
		HPWR		5V			
		HPWR		BATTERY			
		HPWR					

Figure 1-4

C. Power DC interface: The mechArm is powered by a 6.5mm od, 2.0mm OD, and a manufacturer's 8.4V 5A DC power adapter

D. Grove interface: Figure 1-5, Figure 1-6, and Figure 1-7 show the Grove interface definitions

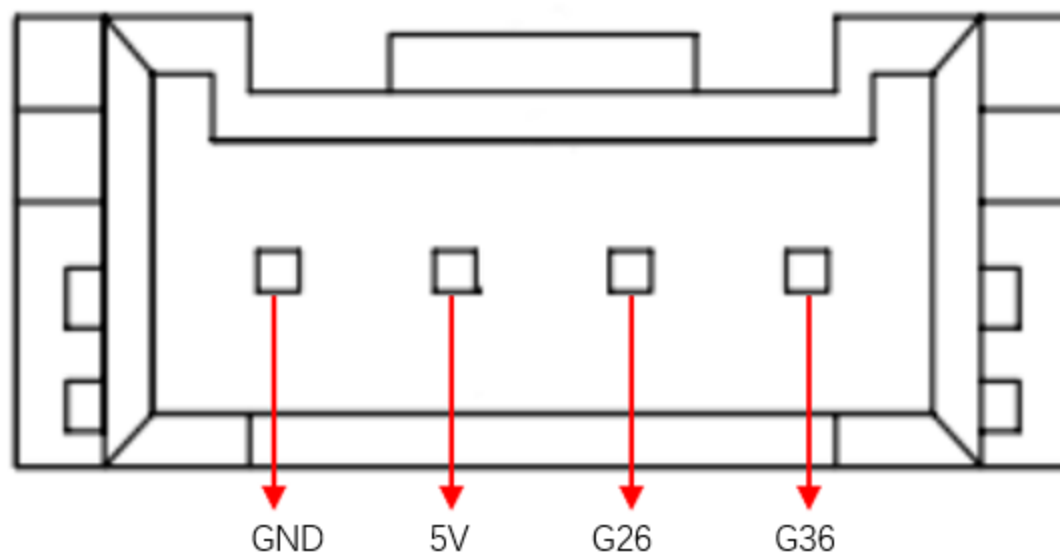


Figure 1-5 Grove interface 1

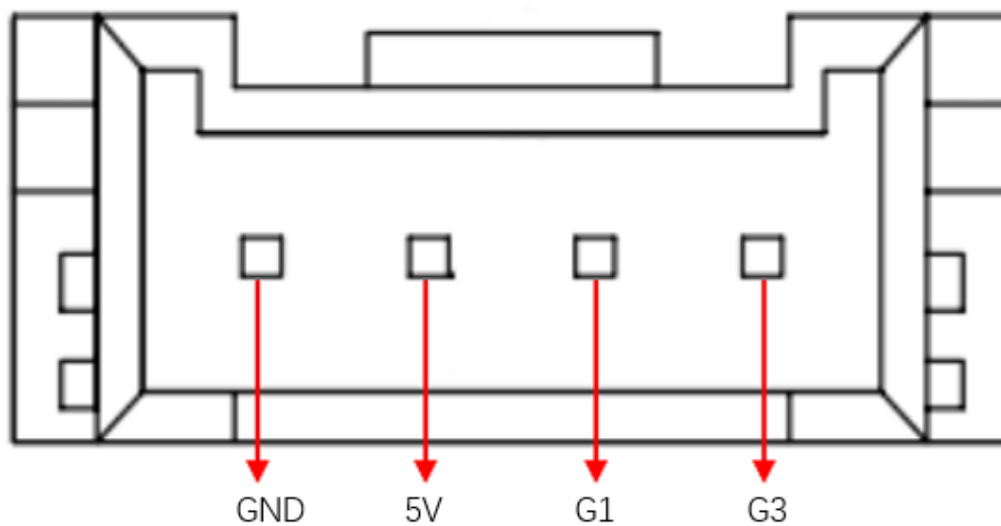


Figure 1-6 Grove interface 2

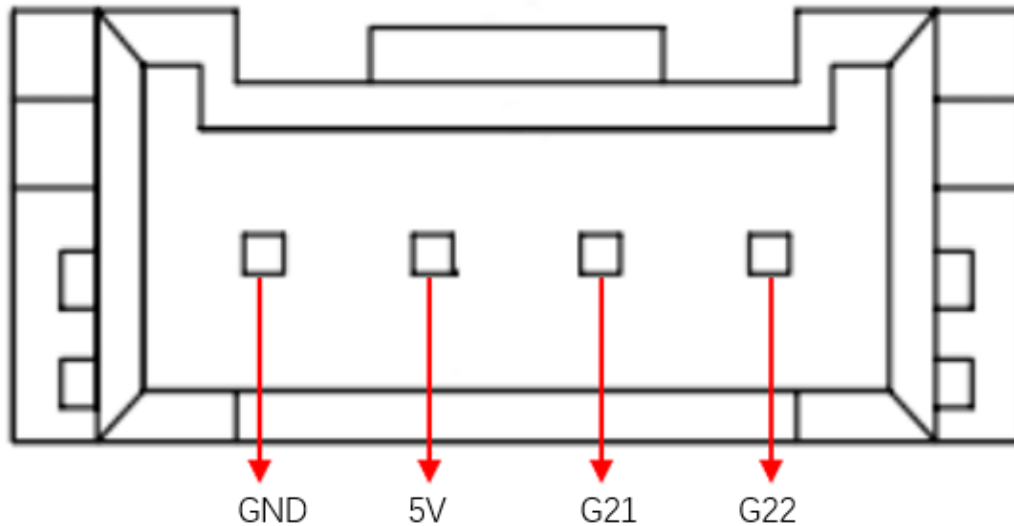


Figure 1-7 Grove interface 3

E. type C interface: used to communicate with the PC. G1 and G3 ports are occupied when this interface is used.

F. Reset button: used when the main control system is reset

G. Buttons A, B, and C: Cooperate with the display

H. Display: The 2-inch IPS screen can be used to display mechArm communication status and adjust robot origin with buttons.

2 Electrical interface at the end of the manipulator

2.1 Introduction of Robot Arm End

A. The side interface of the end of the robot arm is shown in Figure 2-1:

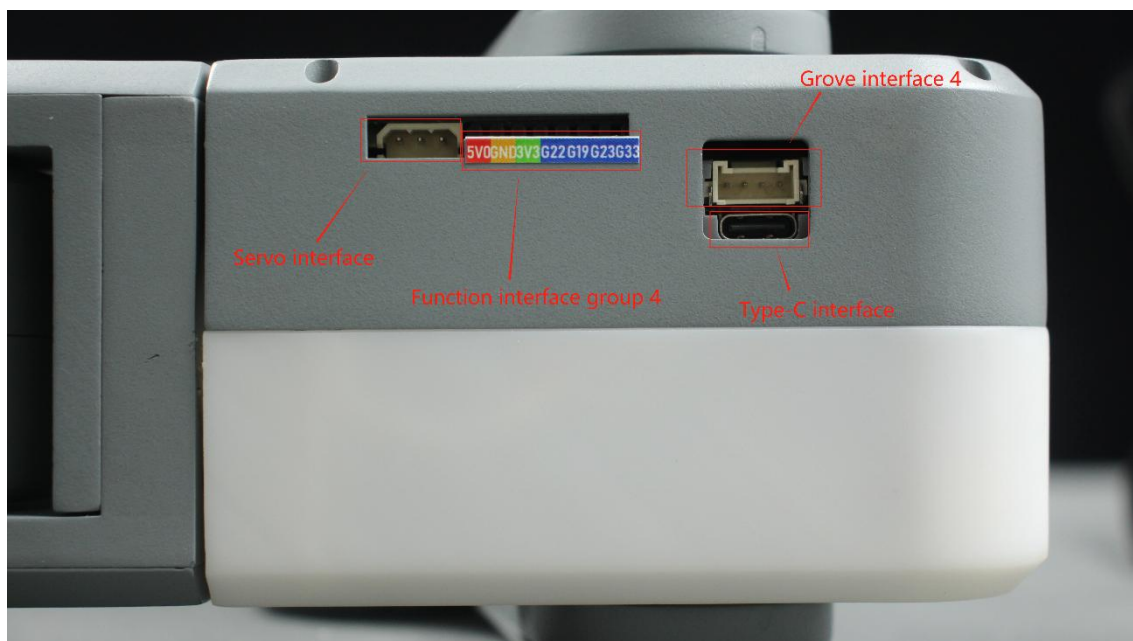


Figure 2-1 End of the robotic arm

2.2 Terminal interface description

A. Table 2-1 shows the definition of each interface in functional interface group 4.

Tag name	Signal name	Function	Note
5V0	5V	Power supply, DC5V	
GND	GND	Motherboard power signal ground	
3V3	3V3	DC3.3 V power supply	
G22	G22	3.3 V-out-PNP output /3.3 V-int input	
G19	G19	3.3 V-out-PNP output /3.3 V-int input	
G23	G23	3.3 V-out-PNP output /3.3 V-int input	

Tag name	Signal name	Function	Note
G33	G33	3.3 V-out-PNP output /3.3 V-int input	

Table 2-1 Functional port group 4

B. Type C interface: used to communicate with PC and update firmware.

C. Grove interface 4: Figure 2-2 shows the definition of Grove interface 4

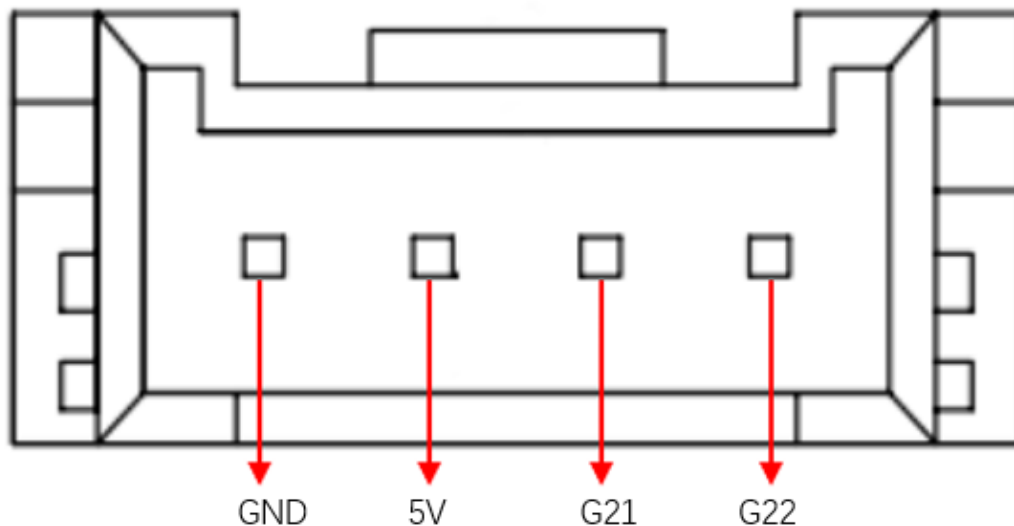


Figure 2-2 Grove interface 4

D. Servo interface: It is used when expanding the gripper at the end and currently supports the use of the matching adaptive gripper.

E. Atom: for 5X5 RGB LED (G27) display and key function (G39)

Python Environment Setup for a mechArm 270 M5 Mounted on a myAGV Jetson Nano 2023

Introduction

pymycobot is the Python API developed by Elephant Robotics for communicating with and controlling the **mechArm 270 M5** through a serial connection.

In this configuration:

- The **mechArm 270 M5** is mounted on the myAGV.
- The **myAGV Jetson Nano 2023** is the main control computer.
- Python programs run directly on the Jetson Nano.
- The mechArm communicates with the Jetson Nano through a USB serial connection.
- The myAGV must remain stationary during the initial communication tests.

The myAGV platform officially supports the mechArm 270 M5 as part of a compound mobile-manipulator configuration, using USB serial communication between the onboard computer and the robotic arm.

Before using pymycobot, complete the following steps:

1. Verify the mechArm 270 M5 firmware.
2. Connect the mechArm to the myAGV Jetson Nano through USB.
3. Identify the serial port assigned to the robotic arm.
4. Configure the Linux serial-port permissions.
5. Install the pymycobot package.
6. Run a communication test.
7. Run the RGB light demonstration.

This guide uses **Python 3**.

1. Safety Preparation

Before configuring or testing the system:

1. Place the myAGV on a flat and stable surface.
2. Make sure the mobile base is not running a navigation or teleoperation program.
3. Stop any ROS node that can send movement commands to the myAGV.
4. Clear the workspace around the robotic arm.
5. Keep people and objects away from the wheels and arm.
6. Verify that all USB and power cables are secured.
7. Make sure the cables cannot interfere with the wheels, LiDAR, or robot joints.

The programs in this guide do not send movement commands to the myAGV base. However, another active program could still move the platform.

2. Preparing the mechArm 270 M5

Before controlling the robotic arm from Python, verify the firmware installed in both controllers.

The mechArm 270 M5 contains:

- An **M5Stack Basic** controller in the base.
- An **Atom** controller near the end of the robotic arm.

2.1 Configure the firmware with MyStudio

Use the Elephant Robotics **MyStudio** application to verify or update the firmware.

The required firmware configuration is:

- **M5Stack Basic:** latest compatible miniRobot firmware.
- **Atom controller:** latest compatible atomMain firmware.

The atomMain firmware is normally installed at the factory. It should be verified or updated when communication problems occur.

MyStudio can be used from a compatible computer before mounting the arm or from the myAGV Jetson Nano when MyStudio is installed and operating correctly.

The myAGV Jetson Nano 2023 system already includes the required environment for its USB serial hardware, so an additional serial driver installation is normally unnecessary.

2.2 Activate USB UART Transponder mode

After turning on the mechArm 270 M5:

1. Use the buttons on the M5Stack Basic.
2. Select **Transponder**.
3. Select **USB UART**.
4. Keep the M5Stack on the Transponder screen.
5. Verify that the display shows:

Atom: OK

Atom: OK indicates that the M5Stack Basic is communicating correctly with the Atom controller.

The USB control procedure requires the M5Stack Basic to remain in **Transponder → USB UART** mode on the Atom: OK screen.

If the screen shows Atom: NO, or if a successful connection is not displayed:

1. Restart the robotic arm.
2. Check the physical connection with the Atom controller.
3. Verify the installed firmware.
4. Reinstall miniRobot and atomMain using MyStudio if necessary.
5. Restart the robotic arm after updating the firmware.

3. Connecting the mechArm to the myAGV

3.1 Power connection

Connect the mechArm 270 M5 to the power connection intended for the robotic arm.

Before turning on the system, verify that:

- The connector is fully inserted.
- The cable is not stretched.
- The cable cannot contact the wheels.
- The cable does not interfere with joint 1.
- The cable does not cross the LiDAR scanning plane.

3.2 USB communication connection

Connect the mechArm 270 M5 directly to the myAGV Jetson Nano:

1. Connect the USB Type-C end of the data cable to the M5Stack Basic controller.
2. Connect the USB end to an available USB port on the myAGV Jetson Nano.
3. Verify that the cable supports data transmission.
4. Secure the cable to prevent it from entering the wheel or joint workspace.

Do not use a charge-only USB cable.

4. Identifying the mechArm Serial Port

The myAGV may have several serial devices connected internally. Therefore, `/dev/ttyUSB0` or `/dev/ttyACM0` should not be selected without verifying which device belongs to the mechArm.

The mechArm 270 M5 normally communicates at **115200 baud**. Official Elephant Robotics examples commonly use `/dev/ttyUSB0`, while other USB configurations may appear as `/dev/ttyACM0`.

4.1 Display the available ports

Open a terminal on the myAGV and run:


```
ls -l /dev/ttyUSB*
```

Then run:

```
ls -l /dev/ttyACM*
```

A serial device may appear as:

/dev/ttyUSB0

or:

/dev/ttyACM0

A message such as the following only means that no device of that type is currently available:

ls: cannot access '/dev/ttyUSB*': No such file or directory

4.2 List detailed USB serial information

Install pyserial if necessary:

```
python3 -m pip install --user --upgrade pyserial
```

Display the connected serial devices:

```
python3 -m serial.tools.list_ports -v
```

This command may show:

- Linux device path.
- Manufacturer.
- Product name.
- Serial number.
- USB vendor ID.
- USB product ID.

4.3 Identify the port by disconnecting and reconnecting the arm

Because the myAGV may have other serial devices, use this procedure:

1. Disconnect only the mechArm USB communication cable.

2. Run:

```
python3 -m serial.tools.list_ports
```

3. Record the devices shown.

4. Reconnect the mechArm USB cable.

5. Run the same command again:

```
python3 -m serial.tools.list_ports
```

6. Identify the new serial device.

The newly appearing port should correspond to the mechArm M5Stack controller.

4.4 Use a persistent serial-port path

Linux may assign a different number after restarting or reconnecting USB devices. For example, `/dev/ttyUSB0` could later become `/dev/ttyUSB1`.

Check whether a persistent device identifier is available:

```
ls -l /dev/serial/by-id/
```

An example result is:

```
/dev/serial/by-id/usb-Silicon_Labs_CP210x_USB_to_UART_Bridge_Controller_...
```

When available, use this path instead of `/dev/ttyUSB0` or `/dev/ttyACM0`.

4.5 Inspect a specific device

To display information about a serial device, run:

```
udevadm info --query=property --name=/dev/ttyUSB0
```

Replace `/dev/ttyUSB0` with the detected device.

Important fields may include:

ID_VENDOR=

ID_MODEL=

ID_SERIAL=

ID_VENDOR_ID=

ID_MODEL_ID=

5. Configuring Serial-Port Permissions

Check the permissions assigned to the detected serial port:

```
ls -l /dev/ttyUSB0
```

Replace /dev/ttyUSB0 with the actual device.

A typical result is:

```
crw-rw---- 1 root dialout ... /dev/ttyUSB0
```

The dialout entry indicates that the user must belong to the dialout group.

5.1 Add the current user to the dialout group

Run:

```
sudo usermod -aG dialout $USER
```

Restart the myAGV:

```
sudo reboot
```

After restarting, verify the user groups:

```
groups
```

The result should include:

```
dialout
```

Do not use the following command as a permanent solution:

```
sudo chmod 777 /dev/ttyUSB0
```

Permissions assigned with chmod are temporary and may be lost after restarting or reconnecting the device.

6. Verifying Python on the myAGV

Open a terminal and run:

```
python3 --version
```


Verify pip:

```
python3 -m pip --version
```

When pip is not installed, use:

```
sudo apt update  
sudo apt install python3-pip
```

7. Installing pymycobot

7.1 Install from PyPI

Install or update pymycobot for the current user:

```
python3 -m pip install --user --upgrade pymycobot
```

Using --user avoids modifying the system Python packages.

7.2 Verify the installation

Run:

```
python3 -m pip show pymycobot
```

If the installation was successful, the package information will be displayed.

Verify the import:

```
python3 -c "from pymycobot import MechArm270;  
print('pymycobot was imported successfully')"
```

Expected output:

pymycobot was imported successfully

The official API uses the MechArm270 class to control the M5 version of the mechArm 270 at 115200 baud.

7.3 Verify which pymycobot version is being used

Run:

```
python3 -c "import pymycobot; print(pymycobot.__file__)"
```

This displays the location of the package used by Python.

To display the installed version:

```
python3 -m pip show pymycobot | grep Version
```

8. Optional Installation from Source Code

Installing from source is normally unnecessary. Use this method only when:

- The standard installation fails.
- A specific GitHub version is required.
- The latest development version is needed.
- The source code must be modified.

Git must be installed:

```
sudo apt update  
sudo apt install git
```

Clone the repository:

```
git clone https://github.com/elephantrobotics/pymycobot.git
```

Open the downloaded directory:

```
cd pymycobot
```

Install the package for the current user:

```
python3 -m pip install --user .
```

For an editable development installation:

```
python3 -m pip install --user -e .
```

To update the repository later:

```
git pull
```

Reinstall the updated package:

```
python3 -m pip install --user --upgrade .
```

Do not maintain both a PyPI installation and an editable source installation unless you know which one Python is importing.

Verify the active installation:

```
python3 -c "import pymycobot; print(pymycobot.__file__)"
```

9. Importing the mechArm 270 M5 API

Use the following import:

```
from pymycobot import MechArm270
```

The MechArm270 object requires:

1. The Linux serial-port path.
2. The baud rate.

Example using /dev/ttyUSB0:

```
mc = MechArm270("/dev/ttyUSB0", 115200)
```

Example using /dev/ttyACM0:

```
mc = MechArm270("/dev/ttyACM0", 115200)
```

Example using a persistent path:

```
mc = MechArm270(  
    "/dev/serial/by-id/<mecharm-device>",  
    115200  
)
```

Replace the serial-port path with the device assigned to the mechArm.

Do not use the myAGV chassis serial port. The MechArm270 object must receive the port associated with the M5Stack Basic controller.

10. Basic Communication Test

Run this test before attempting to move the arm or control the gripper.

Create a file:

nano mecharm_connection_test.py

Enter the following code:

```
#!/usr/bin/env python3

import sys
import time
from pathlib import Path

from pymycobot import MechArm270

BAUD_RATE = 115200

def main():
    """Verify serial communication with the mechArm 270
    M5."""

    if len(sys.argv) != 2:
        print("Usage:")
        print(
            "  python3 mecharm_connection_test.py "
            "<mecharm_serial_port>"
        )
        print("")
        print("Examples:")
        print(
            "  python3 mecharm_connection_test.py "
            "/dev/ttyUSB0"
        )
        print(
            "  python3 mecharm_connection_test.py "
            "/dev/ttyACM0"
        )
        sys.exit(1)
```

```
serial_port = sys.argv[1]

if not Path(serial_port).exists():
    print(f"Serial port not found: {serial_port}")
    print(
        "Run: python3 -m serial.tools.list_ports -v"
    )
    sys.exit(1)

try:
    print("Connecting to the mechArm 270 M5...")
    print(f"Serial port: {serial_port}")
    print(f"Baud rate: {BAUD_RATE}")

    mc = MechArm270(serial_port, BAUD_RATE)

    # Allow time for the serial connection to
initialize.
    time.sleep(2)

    angles = mc.get_angles()

    if isinstance(angles, list) and len(angles) == 6:
        print("Connection successful.")
        print(f"Current joint angles: {angles}")
    else:
        print("The robot did not return six joint
angles.")

        print(f"Received value: {angles}")
        print("")
        print("Verify:")
        print("- Transponder → USB UART is active.")
        print("- The M5Stack displays Atom: OK.")
        print("- The selected port belongs to the
mechArm.")

        print("- No other application is using the
port.")

        sys.exit(1)
```

```
except PermissionError:
    print(f"Permission denied: {serial_port}")
    print("")
    print("Run:")
    print("  sudo usermod -aG dialout $USER")
    print("Then restart the myAGV.")
    sys.exit(1)

except Exception as error:
    print("Communication with the mechArm failed.")
    print(f"Error: {error}")
    sys.exit(1)

if __name__ == "__main__":
    main()
```

Save the file and exit Nano:

Ctrl+O

Enter

Ctrl+X

Run the test:

```
python3 mecharm_connection_test.py /dev/ttyUSB0
```

Replace /dev/ttyUSB0 with the detected port.

A successful response should display a list containing the six joint angles:

Connection successful.

Current joint angles: [J1, J2, J3, J4, J5, J6]

The get_angles() method returns the current angles of the six robotic-arm joints.

11. RGB Light Demonstration

The following program changes the RGB light on the mechArm 270 M5 between blue, red, and green.

The program requires the serial port as a command-line argument. This prevents accidentally using a serial port assigned to another myAGV component.

Create a file:

nano mecharm_rgb_demo.py

Enter the following code:

```
#!/usr/bin/env python3

import sys
import time
from pathlib import Path

from pymycobot import MechArm270

BAUD_RATE = 115200
NUMBER_OF_CYCLES = 7
COLOR_TIME = 2

def main():
    """Change the mechArm RGB light color seven times."""

    if len(sys.argv) != 2:
        print("Usage:")
        print(
            "  python3 mecharm_rgb_demo.py "
            "<mecharm_serial_port>"
        )
        print("")
        print("Examples:")
        print(
            "  python3 mecharm_rgb_demo.py "
            "/dev/ttyUSB0"
        )
```

```
)  
print(  
    "  python3 mecharm_rgb_demo.py "  
    "/dev/ttyACM0"  
)  
sys.exit(1)  
  
serial_port = sys.argv[1]  
  
if not Path(serial_port).exists():  
    print(f"Serial port not found: {serial_port}")  
    print(  
        "Run: python3 -m serial.tools.list_ports -v"  
    )  
    sys.exit(1)  
  
try:  
    print("mechArm 270 M5 RGB test")  
    print(f"Serial port: {serial_port}")  
    print(f"Baud rate: {BAUD_RATE}")  
  
    mc = MechArm270(serial_port, BAUD_RATE)  
  
    # Allow time for serial communication to  
initialize.  
    time.sleep(2)  
  
    # Verify communication before changing the light.  
    angles = mc.get_angles()  
  
    if not isinstance(angles, list) or len(angles) !=  
6:  
        print("The mechArm did not respond correctly.")  
        print(f"Received value: {angles}")  
        print("")  
        print("Verify:")  
        print("- Transponder → USB UART is active.")  
        print("- The M5Stack displays Atom: OK.")
```

```
        print("- The correct serial port was  
selected.")  
        sys.exit(1)  
  
    print(f"Current joint angles: {angles}")  
    print("Communication confirmed.")  
  
    for cycle in range(NUMBER_OF_CYCLES):  
        print(  
            f"RGB cycle {cycle + 1} "  
            f"of {NUMBER_OF_CYCLES}"  
        )  
  
        print("Blue")  
        mc.set_color(0, 0, 255)  
        time.sleep(COLOR_TIME)  
  
        print("Red")  
        mc.set_color(255, 0, 0)  
        time.sleep(COLOR_TIME)  
  
        print("Green")  
        mc.set_color(0, 255, 0)  
        time.sleep(COLOR_TIME)  
  
    # Turn the RGB light off after the test.  
    mc.set_color(0, 0, 0)  
  
    print("RGB test completed successfully.")  
  
except KeyboardInterrupt:  
    print("\nTest interrupted by the user.")  
  
try:  
    mc.set_color(0, 0, 0)  
except Exception:  
    pass
```



```
except PermissionError:
    print(f"Permission denied: {serial_port}")
    print("")
    print("Run:")
    print("  sudo usermod -aG dialout $USER")
    print("Then restart the myAGV.")
    sys.exit(1)

except Exception as error:
    print(
        "Unable to communicate with the "
        "mechArm 270 M5."
    )
    print(f"Error: {error}")
    print("")
    print("Check:")
    print("- The serial-port path.")
    print("- The USB data cable.")
    print("- The firmware versions.")
    print("- Transponder → USB UART.")
    print("- The Atom: OK status.")
    print("- The serial-port permissions.")
    sys.exit(1)

if __name__ == "__main__":
    main()
```

The MechArm270 API provides the `set_color(r, g, b)` method for controlling the RGB light.

11.1 Run the program

Run:

```
python3 mecharm_rgb_demo.py /dev/ttyUSB0
```

Replace /dev/ttyUSB0 with the detected serial port.

The RGB light should change between:

1. Blue.
2. Red.
3. Green.

Each color remains active for two seconds. The sequence is repeated seven times, and the light turns off when the test finishes.

12. Optional Environment Variable

A serial-port environment variable can be used to avoid entering the port repeatedly.

Open the shell configuration file:

```
nano ~/.bashrc
```

Add:

```
export MECHARM_PORT="/dev/ttyUSB0"
```

A persistent path is preferable:

```
export MECHARM_PORT="/dev/serial/by-id/<mecharm-device>"
```

Reload the configuration:

```
source ~/.bashrc
```

Run the connection test:

```
python3 mecharm_connection_test.py "$MECHARM_PORT"
```

Run the RGB demonstration:

```
python3 mecharm_rgb_demo.py "$MECHARM_PORT"
```

13. Troubleshooting

13.1 python3 is not recognized

Verify Python:

which python3

Install Python 3 if necessary:

```
sudo apt update
```

```
sudo apt install python3
```

13.2 pip is not available

Run:

```
python3 -m pip --version
```

If pip is unavailable:

```
sudo apt update
```

```
sudo apt install python3-pip
```

13.3 ModuleNotFoundError: No module named 'pymycobot'

Install the package for the active user:

```
python3 -m pip install --user --upgrade pymycobot
```

Verify the installation:

```
python3 -c "import pymycobot; print(pymycobot.__file__)"
```

Make sure the program is executed with the same Python installation used to install pymycobot.

13.4 No /dev/ttyUSB* device appears

Check /dev/ttyACM*:

```
ls -l /dev/ttyACM*
```

List all serial ports:

```
python3 -m serial.tools.list_ports -v
```

Disconnect and reconnect the mechArm USB cable while comparing the results.

Also verify:

- The mechArm is powered on.
- The USB cable supports data transmission.
- The cable is connected to the M5Stack Basic.
- The USB port on the myAGV is working.
- Transponder → USB UART is active.

13.5 Serial-port permission error

Add the user to the dialout group:

```
sudo usermod -aG dialout $USER
```

Restart the myAGV:

```
sudo reboot
```

13.6 The serial port is busy

An error such as the following indicates that another process may be using the port:

Device or resource busy

Identify the process:

```
sudo fuser -v /dev/ttyUSB0
```

or:

```
sudo lsof /dev/ttyUSB0
```

Replace /dev/ttyUSB0 with the actual port.

Close applications that may be using the robot:

- MyStudio.
- myBlockly.
- Another Python program.
- A ROS mechArm communication node.

- A serial terminal.
- A previously started test program.

Only one program should control the mechArm serial port at a time.

13.7 The wrong serial device is selected

The myAGV may expose other serial ports for its own hardware.

To identify the mechArm:

1. Disconnect only the mechArm USB cable.
2. List the serial ports.
3. Reconnect the cable.
4. List the ports again.
5. Use the newly appearing device.

Do not automatically select the first serial port from a list.

13.8 The mechArm does not respond

Verify:

1. The mechArm 270 M5 is powered on.
2. The USB cable supports data communication.
3. The USB cable is connected to the M5Stack Basic.
4. The selected port belongs to the mechArm.
5. The baud rate is 115200.
6. The M5Stack Basic is running miniRobot.
7. The Atom controller is running atomMain.
8. Transponder → USB UART is active.
9. The screen displays Atom: OK.
10. No other application is using the port.

11. The user belongs to the dialout group.

13.9 The RGB light does not change

Confirm that:

- The communication test returns six joint angles.
- The program does not display a serial error.
- The correct Linux serial port is being used.
- Transponder → USB UART is active.
- The M5Stack Basic displays Atom: OK.
- The installed firmware versions are compatible.
- Another process is not using the serial port.

13.10 The myAGV moves unexpectedly

The programs in this guide do not send chassis commands.

If the myAGV moves:

1. Stop the test program.
2. Stop all active navigation nodes.
3. Stop joystick or keyboard teleoperation.
4. Stop any application controlling the mobile base.
5. Verify that no ROS topic is publishing velocity commands.
6. Repeat the test only after the platform remains stationary.

14. Configuration Summary

Mobile platform:

myAGV Jetson Nano 2023

Robot arm:

mechArm 270 M5

Main computer:

Jetson Nano installed in the myAGV

Operating system:

Linux

Python command:

python3

Python class:

MechArm270

Python import:

```
from pymycobot import MechArm270
```

Communication:

USB serial

Baud rate:

115200

Possible Linux ports:

/dev/ttyUSB0

/dev/ttyUSB1

/dev/ttyACM0

/dev/ttyACM1

Recommended port:

```
/dev/serial/by-id/<mecharm-device>
```

M5Stack Basic firmware:

miniRobot

Atom firmware:

atomMain

Operating mode:

Transponder → USB UART

Required M5Stack status:

Atom: OK

Required Linux group:

dialout

Correct initialization example

```
from pymycobot import MechArm270
```

```
mc = MechArm270("/dev/ttyUSB0", 115200)
```

Replace /dev/ttyUSB0 with the verified mechArm serial port.

Recommended persistent initialization

```
from pymycobot import MechArm270
```

```
mc = MechArm270(  
    "/dev/serial/by-id/<mecharm-device>",  
    115200  
)
```

Do not use a Windows port such as:

```
mc = MechArm270("COM3", 115200)
```

Do not use the internal Raspberry Pi configuration:

```
mc = MechArm270("/dev/ttyAMA0", 1000000)
```

For the mechArm 270 M5 mounted on the myAGV Jetson Nano 2023, use the external USB serial connection at **115200 baud**.

Gripper Control for a mechArm 270 M5 Mounted on a myAGV Jetson Nano

Introduction

This section explains how to control the gripper of a **mechArm 270 M5** from the NVIDIA Jetson Nano computer installed inside a **myAGV Jetson Nano 2023**.

In this configuration:

- The myAGV Jetson Nano is the main computer.
- The mechArm 270 M5 is mounted on the mobile platform.
- The mechArm is powered through the appropriate 12 V power connection.
- The mechArm M5Stack Basic controller is connected to the myAGV through USB.
- Python programs run directly on the myAGV Jetson Nano.
- The gripper is controlled through the pymycobot Python API.

The myAGV Jetson Nano 2023 includes an NVIDIA Jetson Nano B01 and a built-in Linux development environment that supports Python and ROS1 applications.

1. System Configuration

The complete configuration is:

Mobile platform:

myAGV Jetson Nano 2023

Main computer:

NVIDIA Jetson Nano B01

Robot arm:

mechArm 270 M5

End effector:

Adaptive gripper

Communication:

USB serial communication

Python class:

MechArm270

Serial baud rate:

115200

Possible Linux serial ports:

/dev/ttyACM0

/dev/ttyUSB0

/dev/serial/by-id/...

Required M5Stack mode:

Transponder → USB UART

Required arm status:

Atom: OK

The official myAGV documentation supports mounting the mechArm 270 M5 on the mobile platform and controlling it through USB serial communication.

2. Hardware Installation

2.1 Mount the mechArm 270 M5

1. Turn off the myAGV and the mechArm.
2. Place the mechArm 270 M5 on the mounting area of the myAGV.
3. Align the robot base with the corresponding mounting holes.
4. Secure the robot arm using the appropriate screws and mounting hardware.

5. Confirm that the arm cannot rotate or slide relative to the mobile platform.

The arm may be installed toward the front or rear of the platform according to the intended application and workspace.

2.2 Connect the robot power

Connect the appropriate **12 V power cable** between the myAGV and the power input of the mechArm 270 M5.

Verify that:

- The connector is fully inserted.
- The cable is not stretched.
- The cable cannot contact the wheels.
- The cable does not interfere with the movement of joint 1.
- The cable does not cross the LiDAR scanning area.

The official myAGV Jetson Nano example connects the mechArm 270 M5 using a 12 V power cable before starting the system.

2.3 Connect the USB communication cable

Use a data-capable **USB Type-C to USB cable**:

1. Connect the Type-C end to the M5Stack Basic controller at the base of the mechArm.
2. Connect the USB end to an available USB port on the myAGV Jetson Nano.
3. Secure the cable so it does not enter the wheel area.
4. Leave sufficient cable length for the robot arm to rotate safely.

Do not use a charge-only USB cable. The cable must support serial data communication.

2.4 Install the gripper

1. Turn off the system before installing the gripper.
2. Insert the mechanical connector into the gripper mounting socket.

3. Align the gripper with the end of the mechArm.
4. Secure the gripper mechanically.
5. Connect the gripper extension cable.
6. Connect the extension cable to the control interface near the Atom controller.
7. Verify that the cable is not stretched when the wrist moves.

The mechArm 270 M5 supports installing the gripper at the end of the robot and connecting it through the Atom-side control interface.

3. Starting the System

3.1 Safety preparation

Before powering on the compound robot:

1. Place the myAGV on a level surface.
2. Clear the area around the wheels.
3. Clear the workspace around the robot arm.
4. Keep the joystick or emergency stopping method accessible.
5. Do not run navigation or chassis movement programs during the first gripper test.
6. Remove fragile objects from the gripper workspace.

For the first tests, the myAGV must remain stationary.

3.2 Power-on sequence

1. Verify the robot arm mounting.
2. Verify the gripper installation.
3. Verify the 12 V power connection.
4. Verify the USB communication connection.
5. Turn on the myAGV.

6. Wait for the Jetson Nano operating system to start.
7. Verify that the mechArm M5Stack Basic controller turns on.

3.3 Configure the M5Stack Basic

On the M5Stack Basic screen:

1. Open **Transponder**.
2. Select **USB UART**.
3. Keep the controller on the communication screen.
4. Verify that the display shows:

Atom: OK

Atom: OK indicates that the M5Stack Basic is communicating correctly with the Atom controller at the end of the arm.

The official USB procedure for the mechArm 270 M5 requires selecting **Transponder → USB UART** and keeping the robot on the Atom: OK screen.

Do not select WLAN Server for this configuration. The official myAGV documentation lists USB serial communication for the mechArm 270 M5 but does not list this arm under the supported M5 WLAN configuration.

4. Preparing Python on the myAGV Jetson Nano

The myAGV Jetson Nano 2023 includes a built-in development environment, and its system already includes the required USB serial drivers.

Open a terminal on the myAGV.

4.1 Verify Python

Run:

```
python3 --version
```

Verify pip:

```
python3 -m pip --version
```

4.2 Install or update pymycobot

Install pymycobot for the current user:

```
python3 -m pip install --user --upgrade pymycobot
```

Install or update the serial communication package:

```
python3 -m pip install --user --upgrade pyserial
```

Verify the installation:

```
python3 -m pip show pymycobot
```

Test the import:

```
python3 -c "from pymycobot import MechArm270; print('pymycobot is ready')"
```

Expected result:

```
pymycobot is ready
```

5. Identifying the mechArm Serial Port

The Jetson Nano may have several serial devices because the myAGV has its own peripherals. Therefore, do not assume that the first serial port returned by Python belongs to the mechArm.

5.1 List USB serial ports

Run:

```
ls -l /dev/ttyACM*
```

Then:

```
ls -l /dev/ttyUSB*
```

A connected M5Stack may appear as:

```
/dev/ttyACM0
```

or:

```
/dev/ttyUSB0
```

The official documentation specifically shows `/dev/ttyACM0` as a possible mechArm communication port, but the actual name depends on the USB interface and connected devices.

5.2 Use pyserial to list detailed information

Run:

```
python3 -m serial.tools.list_ports -v
```

This displays:

- Device path.
- USB manufacturer.
- Product name.
- Serial number.
- USB vendor and product identifiers.

5.3 Use a persistent device path

When available, list persistent USB paths:

```
ls -l /dev/serial/by-id/
```

Example:

```
/dev/serial/by-id/usb-Silicon_Labs_CP210x_USB_to_UART_Bridge_...
```

A `/dev/serial/by-id/` path is preferable because `/dev/ttyACM0` or `/dev/ttyUSB0` may change after reconnecting devices or restarting the myAGV.

5.4 Confirm the correct port

A practical method is:

1. Disconnect the mechArm USB cable.
2. Run:

```
python3 -m serial.tools.list_ports
```

3. Reconnect the mechArm USB cable.
4. Run the command again.
5. Identify the new device.

Do not use:


```
utils.get_port_list()[0]
```

unless only one serial device is connected. On the myAGV, selecting the first device could select a chassis or peripheral port instead of the mechArm.

6. Configuring Serial-Port Permissions

Check the serial-port permissions:

```
ls -l /dev/ttyACM0
```

Replace /dev/ttyACM0 with the actual port.

If Python displays:

```
PermissionError: [Errno 13] Permission denied: '/dev/ttyACM0'
```

add the current user to the dialout group:

```
sudo usermod -aG dialout $USER
```

Restart the myAGV:

```
sudo reboot
```

After restarting, verify the groups assigned to the current user:

```
groups
```

The output should include:

```
dialout
```

The Elephant Robotics documentation also identifies serial-port permissions as a possible issue when controlling the mechArm from the myAGV Linux system.

7. Gripper Control API

The following functions are available through the MechArm270 class.

7.1 Open or close the gripper

```
mc.set_gripper_state(flag, speed)
```

Parameters:

flag:

0 = open

1 = close

254 = release

speed:

1 to 100

Examples:

```
mc.set_gripper_state(0, 50) # Open
```

```
mc.set_gripper_state(1, 50) # Close
```

```
mc.set_gripper_state(254, 50) # Release
```

7.2 Set an intermediate position

```
mc.set_gripper_value(value, speed)
```

Parameters:

value:

0 to 100

speed:

1 to 100

Example:

```
mc.set_gripper_value(50, 40)
```

7.3 Read the current position

```
value = mc.get_gripper_value()
```

7.4 Check whether the gripper is moving

```
status = mc.is_gripper_moving()
```

Return values:

0 = not moving

1 = moving

-1 = communication or invalid-data error

7.5 Initialize the gripper

```
mc.init_gripper()
```

7.6 Calibrate the gripper

```
mc.set_gripper_calibration()
```

Use calibration only when the gripper is physically positioned at the correct reference point.

The official mechArm 270 API defines a gripper position range of 0–100, a speed range of 1–100, and dedicated functions for opening, closing, reading, initializing, and calibrating the gripper.

8. Communication Test

Create a file:

nano mecharm_connection_test.py

Enter the following code:

```
#!/usr/bin/env python3

import sys
import time
from pathlib import Path

from pymycobot import MechArm270

BAUD_RATE = 115200

def main():
```



```
if len(sys.argv) != 2:
    print("Usage:")
    print("  python3 mecharm_connection_test.py  
<serial_port>")
    print("")
    print("Example:")
    print("  python3 mecharm_connection_test.py  
/dev/ttyACM0")
    sys.exit(1)

serial_port = sys.argv[1]

if not Path(serial_port).exists():
    print(f"Serial port not found: {serial_port}")
    sys.exit(1)

try:
    print(f"Connecting to mechArm 270 M5 through  
{serial_port}...")
    mc = MechArm270(serial_port, BAUD_RATE)

    # Allow the serial connection to initialize.
    time.sleep(2)

    angles = mc.get_angles()

    if angles:
        print("Communication successful.")
        print(f"Current joint angles: {angles}")
    else:
        print("The robot did not return its joint  
angles.")
        print("Verify Transponder → USB UART and Atom:  
OK.")
        sys.exit(1)

except PermissionError:
    print(f"Permission denied for {serial_port}.")
```

```
        print("Add the user to the dialout group and  
restart the myAGV.")  
        sys.exit(1)  
  
    except Exception as error:  
        print("Communication with the mechArm failed.")  
        print(f"Error: {error}")  
        sys.exit(1)  
  
if __name__ == "__main__":  
    main()
```

Save and close the file.

Run it using the detected serial port:

```
python3 mecharm_connection_test.py /dev/ttyACM0
```

A successful response should be similar to:

Connecting to mechArm 270 M5 through /dev/ttyACM0...

Communication successful.

Current joint angles: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]

The reported angles depend on the current robot position.

9. Simple Gripper Demo

Create the file:

```
nano mecharm_gripper_myagv.py
```

Enter the following code:

```
#!/usr/bin/env python3  
  
import sys  
import time  
from pathlib import Path
```

```
from pymycobot import MechArm270

BAUD_RATE = 115200
GRIPPER_SPEED = 40
NUMBER_OF_CYCLES = 5

def wait_for_gripper(mc, timeout=5.0):
    """
    Wait until the gripper stops moving.

    Returns:
        True if the gripper stopped.
        False if a communication error or timeout occurred.
    """

    initial_time = time.monotonic()

    while time.monotonic() - initial_time < timeout:
        status = mc.is_gripper_moving()

        if status == 0:
            return True

        if status == -1:
            print("Unable to read the gripper movement status.")
            return False

        time.sleep(0.1)

    print("The gripper did not stop before the timeout.")
    return False

def main():
```



```
if len(sys.argv) != 2:
    print("Usage:")
    print("  python3 mecharm_gripper_myagv.py  
<serial_port>")
    print("")
    print("Examples:")
    print("  python3 mecharm_gripper_myagv.py  
/dev/ttyACM0")
    print("  python3 mecharm_gripper_myagv.py  
/dev/ttyUSB0")
    sys.exit(1)

serial_port = sys.argv[1]

if not Path(serial_port).exists():
    print(f"Serial port not found: {serial_port}")
    sys.exit(1)

try:
    print("mechArm 270 M5 gripper test")
    print(f"Serial port: {serial_port}")
    print(f"Baud rate: {BAUD_RATE}")

    mc = MechArm270(serial_port, BAUD_RATE)
    time.sleep(2)

    # Verify basic communication before moving the  
gripper.
    angles = mc.get_angles()

    if not angles:
        print("The mechArm did not respond.")
        print("Verify Transponder → USB UART and Atom:  
OK.")
        sys.exit(1)

    print(f"Current joint angles: {angles}")
    print("Communication confirmed.")
```

```
for cycle in range(NUMBER_OF_CYCLES):  
    print(  
        f"\nCycle {cycle + 1} of  
{NUMBER_OF_CYCLES}: "  
        "opening gripper."  
    )  
  
    result = mc.set_gripper_state(0, GRIPPER_SPEED)  
    print(f"Open command result: {result}")  
  
    if not wait_for_gripper(mc):  
        print("Unable to confirm the opening  
movement.")  
        break  
  
    time.sleep(1)  
  
    open_value = mc.get_gripper_value()  
    print(f"Reported open value: {open_value}")  
  
    print(  
        f"Cycle {cycle + 1} of {NUMBER_OF_CYCLES}:  
"  
        "closing gripper."  
    )  
  
    result = mc.set_gripper_state(1, GRIPPER_SPEED)  
    print(f"Close command result: {result}")  
  
    if not wait_for_gripper(mc):  
        print("Unable to confirm the closing  
movement.")  
        break  
  
    time.sleep(1)  
  
    closed_value = mc.get_gripper_value()
```

```
        print(f"Reported closed value: {closed_value}")

# Finish with the gripper open.
print("\nOpening the gripper before finishing.")
mc.set_gripper_state(0, GRIPPER_SPEED)
wait_for_gripper(mc)

print("Gripper test completed.")

except KeyboardInterrupt:
    print("\nTest interrupted by the user.")

    try:
        mc.set_gripper_state(254, GRIPPER_SPEED)
    except Exception:
        pass

except PermissionError:
    print(f"Permission denied for {serial_port}.")
    print("Run:")
    print("  sudo usermod -aG dialout $USER")
    print("Then restart the myAGV.")
    sys.exit(1)

except Exception as error:
    print("Unable to control the mechArm 270 M5 gripper.")
    print(f"Error: {error}")
    print("")
    print("Check:")
    print("- The selected serial port.")
    print("- The USB data cable.")
    print("- Transponder → USB UART.")
    print("- The Atom: OK status.")
    print("- The gripper cable.")
    print("- The pymycobot installation.")
    sys.exit(1)
```



```
if __name__ == "__main__":  
    main()
```

Run the program:

```
python3 mecharm_gripper_myagv.py /dev/ttyACM0
```

Replace /dev/ttyACM0 with the actual port.

The gripper should:

1. Open.
2. Report its position.
3. Close.
4. Report its position.
5. Repeat the sequence five times.
6. Finish in the open position.

10. Intermediate-Position Test

Create the file:

```
nano gripper_position_test.py
```

Enter:

```
#!/usr/bin/env python3  
  
import sys  
import time  
from pathlib import Path  
  
from pymycobot import MechArm270  
  
BAUD_RATE = 115200  
GRIPPER_SPEED = 30
```

```
def main():
    if len(sys.argv) != 2:
        print(
            "Usage: python3 gripper_position_test.py "
            "<serial_port>"
        )
        sys.exit(1)

    serial_port = sys.argv[1]

    if not Path(serial_port).exists():
        print(f"Serial port not found: {serial_port}")
        sys.exit(1)

    try:
        mc = MechArm270(serial_port, BAUD_RATE)
        time.sleep(2)

        if not mc.get_angles():
            print("The mechArm did not respond.")
            sys.exit(1)

        target_values = [0, 25, 50, 75, 100]

        for target in target_values:
            print(f"Sending gripper value: {target}")

            result = mc.set_gripper_value(
                target,
                GRIPPER_SPEED
            )

            print(f"Command result: {result}")
            time.sleep(2)

            measured_value = mc.get_gripper_value()
```

```
        print(f"Reported value: {measured_value}")

    # Return to the open state.
    mc.set_gripper_state(0, GRIPPER_SPEED)
    time.sleep(2)

    print("Position test completed.")

except KeyboardInterrupt:
    print("\nTest interrupted.")

except Exception as error:
    print(f"Gripper test error: {error}")
    sys.exit(1)

if __name__ == "__main__":
    main()
```

Run:

```
python3 gripper_position_test.py /dev/ttyACM0
```

Perform this test without holding an object because the physical opening associated with each value may depend on the gripper installation and calibration.

11. Optional Persistent Configuration

Instead of entering the serial port every time, define an environment variable.

Open .bashrc:

```
nano ~/.bashrc
```

Add:

```
export MECHARM_PORT="/dev/ttyACM0"
```

A persistent /dev/serial/by-id/ path is preferable:


```
export MECHARM_PORT="/dev/serial/by-id/usb-Silicon_Labs_CP210x_USB_to_UART_Bridge_..."
```

Reload the file:

```
source ~/.bashrc
```

Run the program with:

```
python3 mecharm_gripper_myagv.py "$MECHARM_PORT"
```

12. Troubleshooting

12.1 No serial port appears

Run:

```
python3 -m serial.tools.list_ports -v
```

Then verify:

- The USB cable supports data communication.
- The cable is connected to the M5Stack Basic.
- The mechArm is powered on.
- The M5Stack is in USB UART mode.
- The USB connector is not loose.
- Another USB port on the myAGV works correctly.

The myAGV Jetson Nano 2023 system should not normally require an additional M5Stack USB driver installation.

12.2 Permission denied

Run:

```
sudo usermod -aG dialout $USER
```

```
sudo reboot
```

Do not use sudo to run every robot program as the permanent solution.

12.3 The wrong serial device is selected

Disconnect and reconnect only the mechArm USB cable while comparing:

```
python3 -m serial.tools.list_ports -v
```

Use the corresponding /dev/serial/by-id/ path when possible.

12.4 The robot does not return its joint angles

Verify:

Transponder → USB UART

Atom: OK

Baud rate: 115200

Also close any program that could be using the serial port:

- myStudio.
- myBlockly.
- Another Python program.
- A ROS mechArm driver.
- A serial terminal.

Only one process should control the mechArm serial port at a time.

12.5 The gripper does not move

Check:

- The gripper mechanical installation.
- The extension cable.
- The Atom-side connector.
- The Atom firmware.
- The selected gripper type.
- The serial communication.
- The Atom: OK status.

Try initializing the gripper:

```
result = mc.init_gripper()
```

```
print(result)
```

```
time.sleep(2)
```

Then open it slowly:

```
mc.set_gripper_state(0, 20)
```

12.6 The gripper applies excessive force

Release the gripper:

```
mc.set_gripper_state(254, 20)
```

Then open it:

```
mc.set_gripper_state(0, 20)
```

Reduce the testing speed and do not use fragile objects until the gripper behavior has been verified.

12.7 The myAGV moves unexpectedly

The gripper test does not send movement commands to the mobile base. If the myAGV moves:

1. Stop the program.
2. Stop any active ROS navigation or teleoperation node.
3. Close AGV control applications.
4. Verify that the joystick is not sending commands.
5. Perform the test again with no chassis-control program running.

13. API and Connection Summary

Computer:

myAGV Jetson Nano 2023

Robot:

mechArm 270 M5

Gripper:

Adaptive gripper

Python import:

```
from pymycobot import MechArm270
```

Serial initialization:

```
mc = MechArm270("/dev/ttyACM0", 115200)
```

Recommended persistent port:

```
/dev/serial/by-id/...
```

Open:

```
mc.set_gripper_state(0, speed)
```

Close:

```
mc.set_gripper_state(1, speed)
```

Release:

```
mc.set_gripper_state(254, speed)
```

Set position:

```
mc.set_gripper_value(value, speed)
```

Read position:

```
mc.get_gripper_value()
```

Check movement:

```
mc.is_gripper_moving()
```

Initialize:

```
mc.init_gripper()
```

Calibrate:

```
mc.set_gripper_calibration()
```

Gripper value range:

0 to 100

Speed range:

1 to 100

M5Stack mode:

Transponder → USB UART

Required display:

Atom: OK